

子午线弧长计算及大地纬度反算报告

一、运行方式

运行环境：VS2022 C#

可运行程序文件：子午线弧长计算及大地纬度反算代码\bin\Debug\

GeodeticCalculation.exe

执行方式：

1. 输入 P1、P2 两点的经纬度（大地坐标）；
2. 点击开始计算，查看结果。

二、原理

1. 子午线弧长公式：

子午线弧长计算基于地球椭球模型，通过对子午圈曲率半径 M 积分得到赤道到某纬度 B 的弧长 X ，基本积分式

$$X = \int_0^B M dB$$

其中 M 经级数展开、积分整理后有多种形式

$$X = a_0 B - \frac{a_2}{2} \sin 2B + \frac{a_4}{4} \sin 4B - \dots$$

a_0, a_2 等为与椭球参数相关的系数，不同椭球（如北京 54 坐标系基于克拉索夫斯基椭球）代入对应参数可计算具体弧长。

2. 由子午线弧长反算大地纬度

2.1 直接法:

利用迭代公式，以弧长 X 结合椭球参数，通过多次迭代逼近大地纬度 B ，如对克拉索夫斯基椭球，迭代式基于子午线弧长公式变形，不断修正纬度初值直至满足精度要求（ $|B_{j+1} - B_j| < \varepsilon$ ， ε 为给定精度，如 5×10^{-10} ）。

2.2 间接法:

借助三角级数回代公式，将弧长与纬度的关系通过级数形式表达，代入弧长计算纬度，公式形式如

$$B = \beta + P_2 \sin 2x + P_4 \sin 4x + \dots$$

β 为初值， P_2, P_4 等为系数。

3. 两点间子午线弧长

3.1 直接法（单点展开）:

对接近的两点 $P_1(B_1)$ 、 $P_2(B_2)$ ，将 $\Delta X = X_2 - X_1$ 在 P_1 点按泰勒级数展开

$$\Delta X = \left(\frac{dX}{dB} \right)_{B_1} \Delta B + \frac{1}{2} \left(\frac{d^2X}{dB^2} \right)_{B_1} \Delta B^2 + \dots$$

利用子午圈曲率半径 M 及其导数与 ΔB （纬度变化量 $\Delta B = B_2 - B_1$ ）关联计算。

3.2 直接法（中点展开）：

在 $B_m = \frac{B_1+B_2}{2}$ 点展开级数，可减少项数，公式形式与单点展开类似，基于中点处的 M 及导数计算 ΔX 。

3.3 间接法：

分别计算两点到赤道的子午线弧长 X_1, X_2 ，两点间弧长 $\Delta X = X_2 - X_1$ ，纬度变化量 $\Delta B = B_2 - B_1$ 可通过反算纬度后求差，或利用弧长与纬度的函数关系直接推导。

4. 利用两点间弧长反算纬度变化量

两点纬度为 B_1, B_2 ，对应弧长 X_1, X_2 ，则弧长差 $\Delta X = X_2 - X_1$ 。利用正算级数式，将 X_2, X_1 分别展开为 B_2, B_1 的级数，作差后得到 ΔX 与 ΔB 的关系：

$$\Delta X = M_m \Delta B + \text{高阶小项}$$

M_m 为平均纬度处的子午圈曲率半径，简化时可近似为线性关系 $\Delta X \approx M_m \Delta B$ 。

先假设初始纬度 $B_j^{(0)}$ ，代入正算公式计算弧长 $X_j^{(0)}$ ；对比已知弧长 ΔX ，通过修正项（如含 $\sin 2B$ 、 $\sin 4B$ 等三角级数）迭代更新纬度 $B_j^{(n+1)}$ ，直到 $|B_j^{(n+1)} - B_j^{(n)}| < \varepsilon$ （ $\varepsilon = 5 \times 10^{-10}$ ）本质是利用正算公式的逆过程，通过逐步逼近反推纬度差。

小弧长时，可忽略高阶项，用线性近公式 $\Delta B \approx \frac{\Delta X}{M_m}$ 计算。

三、代码

```
using System;
using System.Windows.Forms;
using System.Globalization;
using System.Drawing;
namespace GeodeticMeridianCalculation
{
    public partial class Form1 : Form
    {
        // 椭球参数（北京 54 坐标系）
        private const double A = 6378245;
        private const double Alpha = 1 / 298.3;
        private readonly double E2 = (2 * Alpha - Alpha * Alpha) / (1 + Alpha);
        private const double A0 = 111134.861;
        private const double C = 6367588.4969;
        private const double Epsilon = 1e-10;
        public Form1()
        {
            InitializeComponent();
            SetExampleValues();
        }
        private void SetExampleValues()
        {
            txtB1.Text = "30";    // P1 纬度
            txtL1.Text = "181°45'36\"";    // P1 经度（仅显示）
            txtB2.Text = "30.5"; // P2 纬度
            txtL2.Text = "181°45'36\""; // P2 经度（仅显示）
        }
        private void btnCalculate_Click(object sender, EventArgs e)
        {
            try
            {
                // 解析输入
                double b1 = ParseCoordinate(txtB1.Text, "纬度 P1");
                double b2 = ParseCoordinate(txtB2.Text, "纬度 P2");
                // 第一部分：赤道到点的弧长
                string part1 = GetPart1Result(b1, b2);
                // 第二部分：反算纬度（直接法/间接法）
                string part2 = GetPart2Result(b1, b2);
                // 第三部分：两点间弧长及纬度差
                string part3 = GetPart3Result(b1, b2);
                // 分区块显示结果
                rtbResult.Clear();
            }
            catch { }
        }
    }
}
```

```

        ShowSection("一、赤道到点的子午线弧长", part1, Color.Teal);
        ShowSection("二、由弧长反算大地纬度", part2, Color.Navy);
        ShowSection("三、两点间子午线弧长与纬度差", part3, Color.Maroon);
    }
    catch (Exception ex)
    {
        rtbResult.ForeColor = Color.Red;
        rtbResult.Text = $"错误: {ex.Message}";
    }
}

// 第一部分: 赤道到点的弧长
private string GetPart1Result(double b1, double b2)
{
    double x1 = CalculateMeridianArcLength(b1);
    double x2 = CalculateMeridianArcLength(b2);
    return $"P1 (B={b1:F4}°) : {x1:F3} m
P2 (B={b2:F4}°) : {x2:F3} m";
}

// 第二部分: 反算纬度
private string GetPart2Result(double b1, double b2)
{
    double x1 = CalculateMeridianArcLength(b1);
    double x2 = CalculateMeridianArcLength(b2);
    // 直接法 (迭代法)
    double directB1 = InverseLatitudeDirect(x1);
    double directB2 = InverseLatitudeDirect(x2);
    // 间接法 (级数展开)
    double indirectB1 = InverseLatitudeIndirect(x1);
    double indirectB2 = InverseLatitudeIndirect(x2);
    return $"[直接法 (迭代法)]
P1 反算: {directB1:F8}° (误差: {Math.Abs(directB1 - b1):F10}°)
P2 反算: {directB2:F8}° (误差: {Math.Abs(directB2 - b2):F10}°)
[间接法 (级数展开)]
P1 反算: {indirectB1:F8}° (误差: {Math.Abs(indirectB1 - b1):F10}°)
P2 反算: {indirectB2:F8}° (误差: {Math.Abs(indirectB2 - b2):F10}°) ";
}

// 第三部分: 两点间弧长与纬度差
private string GetPart3Result(double b1, double b2)
{
    double deltaB = b2 - b1;
    double deltaXDirectP1 = DeltaXDirectFromP1(b1, deltaB);
    double deltaXDirectMid = DeltaXDirectFromMid(b1, b2);
    double deltaXIndirect = CalculateMeridianArcLength(b2) - CalculateMeridianArcLength(b1);
    // 新增: 由弧长反算纬度差 (间接法)

```

```

        double deltaB_FromX = InverseDeltaB(deltaXIndirect);
        return $"[直接法（起点 P1 展开）]";
    }

    ΔX = {deltaXDirectP1:F3} m
[直接法（中点 Bm 展开）]
    ΔX = {deltaXDirectMid:F3} m
[间接法（X2-X1）]
    ΔX = {deltaXIndirect:F3} m
[反算纬度差]
    ΔB: {deltaB_FromX:F6}°";
}

// 由弧长反算纬度差（用于第三部分）
private double InverseDeltaB(double deltaX)
{
    double x1 = 0; // 赤道到赤道的弧长为 0
    double x2 = deltaX;
    double b2 = InverseLatitudeDirect(x2);
    return b2 - 0; // 从赤道反算，故 ΔB = b2 - 0
}

// 计算子午线弧长
private double CalculateMeridianArcLength(double bDeg)
{
    double bRad = bDeg * Math.PI / 180;
    return A0 * bDeg - 16036.480 * Math.Sin(2 * bRad) + 16.828 * Math.Sin(4 * bRad) - 0.022 * Math.Sin(6
* bRad);
}

// 直接法反算纬度（迭代法）
private double InverseLatitudeDirect(double x)
{
    double betaRad = x / C;
    double bCurr = betaRad, bPrev = 0;
    for (int i = 0; i < 100; i++)
    {
        bPrev = bCurr;
        double sin2B = Math.Sin(2 * bPrev);
        double sin4B = Math.Sin(4 * bPrev);
        double sin6B = Math.Sin(6 * bPrev);
        bCurr = (x + 32005.780 * sin2B - 135.3303 * sin4B + 0.7092 * sin6B) / A0 * Math.PI / 180;
        if (Math.Abs(bCurr - bPrev) < Epsilon) break;
    }
    return bCurr * 180 / Math.PI;
}

// 间接法反算纬度（级数展开）
private double InverseLatitudeIndirect(double x)
{

```

```

        double betaDeg = x / A0;
        double betaRad = betaDeg * Math.PI / 180;
        double p2 = 2.518828e-3, p4 = 3.701007e-6;
        return (betaRad + p2 * Math.Sin(2 * betaRad) + p4 * Math.Sin(4 * betaRad)) * 180 / Math.PI;
    }

    // 直接法（起点展开）
    private double DeltaXDirectFromPl(double b1Deg, double deltaBDeg)
    {
        double b1Rad = b1Deg * Math.PI / 180;
        double deltaBRad = deltaBDeg * Math.PI / 180;
        double M1 = A * (1 - E2) / Math.Pow(1 - E2 * Math.Sin(b1Rad) * Math.Sin(b1Rad), 1.5);
        double term = (3.0 / 2) * A * E2 * (1 - E2) * deltaBRad * deltaBRad / 2 *
            (1 + 5.0 / 2 * E2 * Math.Sin(b1Rad) * Math.Sin(b1Rad)) * Math.Sin(2 * b1Rad);
        return M1 * deltaBRad + term + (3.0 / 2 * A * E2 * (1 - E2) * Math.Cos(2 * b1Rad) * Math.Pow(deltaBRa
d, 3)) / 3;
    }

    // 直接法（中点展开）
    private double DeltaXDirectFromMid(double b1Deg, double b2Deg)
    {
        double bmDeg = (b1Deg + b2Deg) / 2;
        double bmRad = bmDeg * Math.PI / 180;
        double deltaBRad = (b2Deg - b1Deg) * Math.PI / 180;
        double Mm = A * (1 - E2) / Math.Pow(1 - E2 * Math.Sin(bmRad) * Math.Sin(bmRad), 1.5);
        double term = (A * E2 * (1 - E2) / 8) * Math.Cos(2 * bmRad) * deltaBRad * deltaBRad;
        return Mm * deltaBRad + term;
    }

    // 坐标解析
    private double ParseCoordinate(string input, string type)
    {
        if (!double.TryParse(input, out double value))
            throw new ArgumentException($" {type} 格式错误");
        if (type.Contains("纬度") && (value < -90 || value > 90))
            throw new ArgumentException($" {type} 必须在[-90°, 90°]之间");
        return value;
    }

    // 分区块显示结果
    private void ShowSection(string title, string content, Color color)
    {
        rtbResult.SelectionColor = Color.Black;
        rtbResult.AppendText(Environment.NewLine);
        rtbResult.SelectionColor = color;
        rtbResult.AppendText($"===== {title} =====\n");
        rtbResult.SelectionColor = Color.Black;
        rtbResult.AppendText(content + Environment.NewLine);
    }

```

```

    }

    private void btnClear_Click(object sender, EventArgs e)
    {
        txtB1.Clear();
        txtL1.Clear();
        txtB2.Clear();
        txtL2.Clear();
        rtbResult.Clear();
        SetExampleValues();
    }

    private void label1_Click(object sender, EventArgs e)
    {
    }
}
}

```

四、结果

初始界面：

子午线弧长计算及大地纬度反算 - 张洋20232411

输入北京54坐标系坐标

请输入P1点经纬度：

经度P1 181°45'36"

纬度P1 30

请输入P2点经纬度：

经度P2 181°45'36"

纬度P2 30.5

开始计算

清空并恢复示例

计算结果示例：

子午线弧长计算及大地纬度反算 - 张洋20232411

输入北京54坐标系坐标

请输入P1点经纬度：

经度P1181°45'36"

纬度P130

请输入P2点经纬度：

经度P2181°45'36"

纬度P230.5

开始计算

清空并恢复示例

==== 一、赤道到点的子午线弧长 ====

P1 (B=30.0000°) : 3320172.404 m

P2 (B=30.5000°) : 3375601.711 m

==== 二、由弧长反算大地纬度 ====

[直接法 (迭代法)]

P1反算: 30.12414523° (误差: 0.1241452305°)

P2反算: 30.62538641° (误差: 0.1253864087°)

[间接法 (级数展开)]

P1反算: 30.00001803° (误差: 0.0000180303°)

P2反算: 30.50001823° (误差: 0.0000182314°)

==== 三、两点间子午线弧长与纬度差 ====

[直接法 (起点P1展开)]

$\Delta X = 55430.082\text{ m}$

子午线弧长计算及大地纬度反算 - 张洋20232411

输入北京54坐标系坐标

请输入P1点经纬度：

经度P1181°45'36"

纬度P130

请输入P2点经纬度：

经度P2181°45'36"

纬度P230.5

开始计算

清空并恢复示例

[间接法 (级数展开)]

P1反算: 30.00001803° (误差: 0.0000180303°)

P2反算: 30.50001823° (误差: 0.0000182314°)

==== 三、两点间子午线弧长与纬度差 ====

[直接法 (起点P1展开)]

$\Delta X = 55430.082\text{ m}$

[直接法 (中点Bm展开)]

$\Delta X = 55430.278\text{ m}$

[间接法 (X_2-X_1)]

$\Delta X = 55429.307\text{ m}$

[反算纬度差]

$\Delta B: 0.503779^\circ$